

S2: Autotyping Challenge

Build a ROS 2 node that autonomously types a launch key on a simulated keyboard. The challenge is based on the autonomous typing task of the 2026 University Rover Challenge.

The simulator is provided: [AutoTypingChallengeSim](#). It gives you a five-joint, velocity-controlled arm with a forearm camera, facing a keyboard on a panel marked with ArUco markers. Start with docs/INFO.md for setup; docs/INTERFACES.md is the complete reference.

Your node must be your own implementation and may interact with the simulator only through the ROS topics and services in INTERFACES.md. The dashboard, teleop and debug_press_feedback may be used for debugging, but the final attempt must run fully autonomously with teleop and debug_press_feedback off.

Remember this is designed to be an *Impossible* challenge. Even if you barely finish a portion of the required functionality, you can still have a great presentation if you are able to adequately explain what you learned. Quality over quantity applies a lot here.

Required Functionality

Your system must:

- Read the launch key from /sim/launch_key.
- Control the arm in closed loop from /joint_states, within its joint and velocity limits.
- Determine the panel's position and orientation from the camera image.
- Determine where the keys of the launch key are. Key positions are not provided.
- Move the arm to a pose from which every character of the launch key can be pressed.
- Aim at and press each character in order, then publish /sim/done.

- Type the launch key exactly, so /sim/result reports an exact match.
- Work on any episode seed without code changes. The panel's placement changes between episodes.
- Record detections, pose estimates, commands and press decisions in an exportable log.

ROS Requirements

Your solution should demonstrate proper use of:

- ROS topics and services
- Coordinate transforms
- Appropriate frame IDs
- ROS timestamps
- QoS settings
- DDS-based communication
- Separation between perception, planning and arm control
- Safe behavior when nodes, joint states or camera data become unavailable

Assume real camera data and arm motion are less ideal than the simulator's. You do not need to simulate this, but your design should account for it.

Repository, Log, and Presentation

Push all code, launch files and setup instructions to a public GitHub repository.

In your presentation, explain:

- How the panel's position and orientation are estimated, and how accurate the estimate is
- How key positions are determined
- How the camera, arm and panel frames are related
- How the typing pose is chosen
- How the arm is controlled, and how your node decides it is safe to press
- What happens during stale data or node failure
- What would need to change for real rover hardware

- What remains unfinished
- Type the launch key correctly on 10 consecutive episodes, advancing with /sim/reset, without restarting or retuning your node.
- Write a URDF for the arm from the numbers in INTERFACES.md and use MoveIt 2 to plan arm motion. Execute its plans through the velocity interface.
- Minimize the time from launch to /sim/done while keeping an exact match. Report your elapsed times across several seeds.
- The simulator and your node launching from your documented setup
- Marker detection and your panel estimate, shown in your own visualization
- The arm moving to a pose from which the launch key is reachable
- The full launch key typed autonomously, with the dashboard's result panel showing the outcome
- The same code succeeding on a second seed you did not tune for
- A failure case: lost camera data, an unreachable key, a rejected press, or another fault
- Exported logs

Your presentation should not be longer than 20 mins.

Review the [official URC 2026 requirements](#) and [URC 2026 Q&A](#). You are responsible for identifying which autonomous-typing rules and field constraints should influence your design.

Note:

AI use for code is encouraged here and you should be able to explain how you used AI in the coding process.

****IMPORTANT****

This does not mean that you can have code in the repository that you cannot explain. We will be grilling you during presentations so be ready to explain every part of your system. AI is a tool, not something to rely completely on you won't have access to it during competition so KNOW YOUR STUFF.